

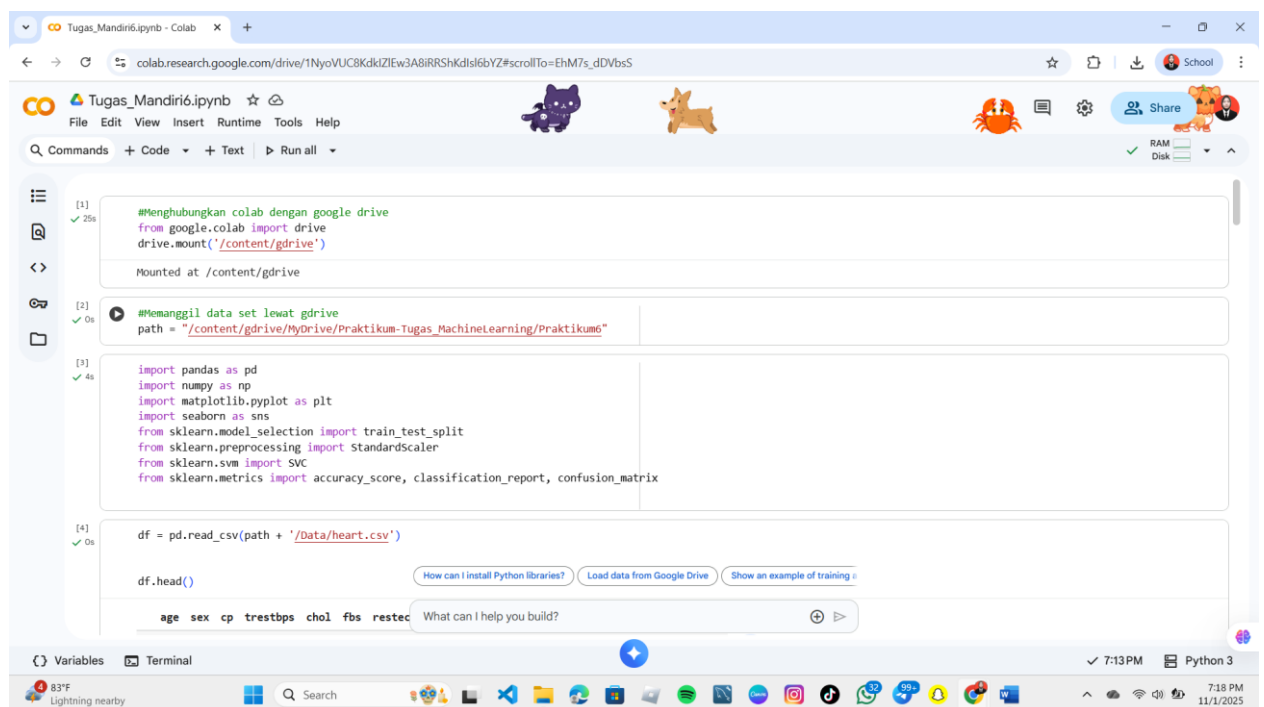
Tugas 6: Tugas Praktikum Mandiri 6 – Machine Learning

Yurida Yahsyia 1 - 0110224100 ^{1*}

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 0110224100@student.nurulfikri.ac.id – email mahasiswa 1

1. Latihan Mandiri 6



1. **from google.colab import drive**

Memanggil modul bawaan Colab.

1. **drive.mount('/content/gdrive')**

Membuat “jembatan” supaya Colab bisa membaca file yang ada di Drive, dan semua file bisa diakses lewat folder /content/gdrive/MyDrive/.

2. **path = "/content/gdrive/MyDrive/Praktikum-Tugas_MachineLearning/Praktikum6"** path = alamat menuju folder Praktikum6 di dalam Google Drive.

3. **import pandas as pd**

Mengimpor library Pandas untuk bekerja dengan DataFrames.

4. **import numpy as np**

Mengimpor library NumPy untuk operasi numerik.

5. **import matplotlib.pyplot as plt**

Mengimpor modul Pyplot dari Matplotlib untuk visualisasi data.

6. import seaborn as sns

Mengimpor library Seaborn untuk visualisasi statistik (digunakan untuk heatmap korelasi dan confusion matrix).

7. from sklearn.model_selection import train_test_split

Mengimpor fungsi train_test_split untuk membagi data latih dan data uji.

8. from sklearn.preprocessing import StandardScaler

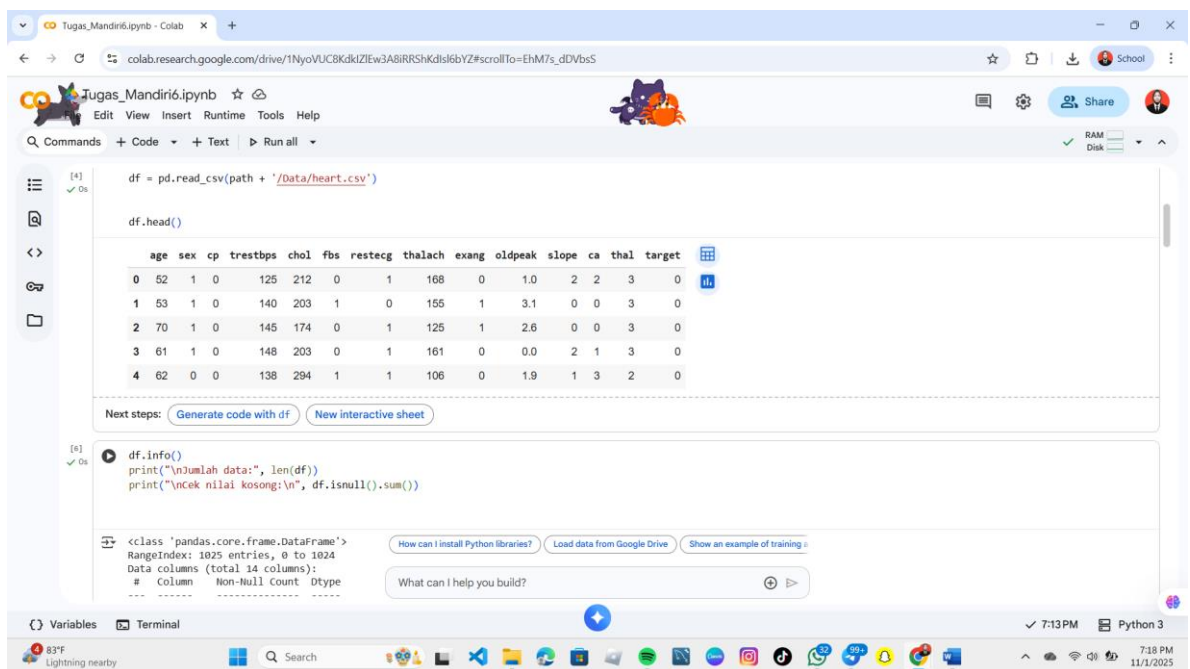
Mengimpor StandardScaler untuk menstandarisasi fitur.

9. from sklearn.svm import SVC

Mengimpor Support Vector Classifier (SVC), yaitu algoritma SVM untuk klasifikasi.

10. from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

Mengimpor metrik evaluasi yang digunakan untuk mengukur kinerja model.



The screenshot shows a Google Colab notebook titled 'Tugas_Mandiri6.ipynb'. The first code cell contains the following Python code:

```
df = pd.read_csv(path + '/Data/heart.csv')
df.head()
```

The output of the second cell shows the first five rows of the 'heart.csv' dataset:

	age	sex	cp	trestbps	chol	fb	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

The second code cell contains the following Python code:

```
df.info()
print("\nJumlah data:", len(df))
print("\nCek nilai kosong:", df.isnull().sum())
```

The output of the second cell shows the following information:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 # Column Non-Null Count Dtype
---
0 age 1025 non-null int64
1 sex 1025 non-null int64
2 cp 1025 non-null int64
3 trestbps 1025 non-null int64
4 chol 1025 non-null int64
5 fbs 1025 non-null int64
6 restecg 1025 non-null int64
7 thalach 1025 non-null int64
8 exang 1025 non-null int64
9 oldpeak 1025 non-null float64
10 slope 1025 non-null int64
11 ca 1025 non-null int64
12 thal 1025 non-null int64
13 target 1025 non-null int64
dtypes: int64(13), float64(1)
memory usage: 102.5 KB
```

1. df = pd.read_csv(path + '/Data/heart.csv')

Kode ini menggunakan library Pandas (pd) untuk membaca file heart.csv dari jalur (path) yang telah ditentukan sebelumnya (lokasi di Google Drive). Hasil pembacaan file disimpan dalam variabel DataFrame df.

2. df.head()

Fungsi ini menampilkan 5 baris pertama dari DataFrame df. Hal ini dilakukan untuk melihat sekilas data, memastikan data termuat dengan benar, dan memeriksa nama-nama kolom beserta tipe datanya.

Output: Tabel yang menampilkan baris 0 hingga 4, menunjukkan kolom-kolom (fitur) yang digunakan, seperti age, sex, cp (chest pain type), trestbps (resting blood pressure), chol (kolesterol), hingga kolom target (target).

```
df.info()
print("\nJumlah data:", len(df))
print("\nCek nilai kosong:\n", df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   age         1025 non-null   int64  
 1   sex         1025 non-null   int64  
 2   cp          1025 non-null   int64  
 3   trestbps    1025 non-null   int64  
 4   chol        1025 non-null   int64  
 5   fbs         1025 non-null   int64  
 6   restecg     1025 non-null   int64  
 7   thalach     1025 non-null   int64  
 8   exang       1025 non-null   int64  
 9   oldpeak     1025 non-null   float64 
10   slope       1025 non-null   int64  
11   ca          1025 non-null   int64  
12   thal        1025 non-null   int64  
13   target      1025 non-null   int64  
dtypes: float64(1), int64(13)
memory usage: 112.2 KB

Jumlah data: 1025

Cek nilai kosong:
age          0
sex          0
```

```
4   chol        1025 non-null   int64  
5   fbs         1025 non-null   int64  
6   restecg     1025 non-null   int64  
7   thalach     1025 non-null   int64  
8   exang       1025 non-null   int64  
9   oldpeak     1025 non-null   float64 
10   slope       1025 non-null   int64  
11   ca          1025 non-null   int64  
12   thal        1025 non-null   int64  
13   target      1025 non-null   int64  
dtypes: float64(1), int64(13)
memory usage: 112.2 KB

Jumlah data: 1025

Cek nilai kosong:
age          0
sex          0
cp           0
trestbps     0
chol         0
fbs          0
restecg      0
thalach      0
exang        0
oldpeak      0
slope        0
ca           0
thal         0
target       0
dtype: int64
```

1. df.info()

fungsi dari library Pandas yang menampilkan ringkasan teknis DataFrame df. Ini sangat penting untuk memverifikasi tipe data dan melihat status missing values secara cepat.

2. print("\nJumlah data:", len(df))

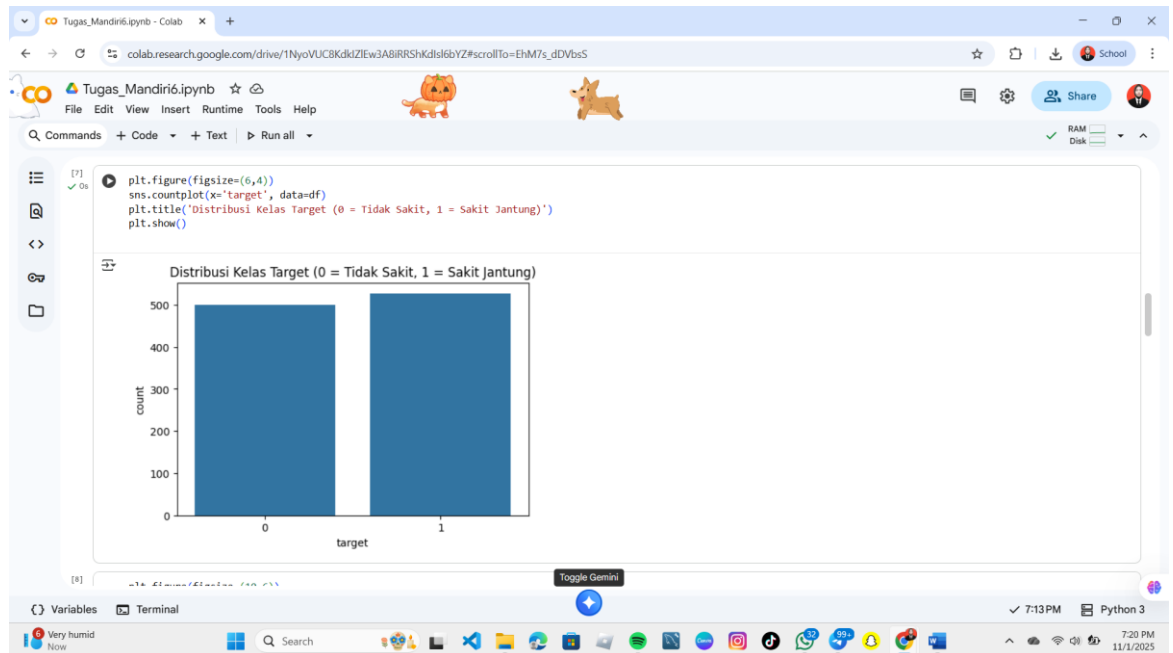
Fungsi len(df) menghitung total jumlah baris (sampel data) dalam DataFrame. Hasilnya ditampilkan dalam kalimat informatif.

3. print("\nCek nilai kosong:\n", df.isnull().sum())

Ini untuk memeriksa missing values. Fungsi df.isnull() membuat DataFrame boolean (True/False), dan .sum() menghitung jumlah nilai True (nilai kosong) di setiap kolom.

Output:

dari kode tersebut mengonfirmasi bahwa data bersih dan siap untuk scaling dan pemodelan, dengan rincian seperti digambar.



1. **plt.figure(figsize=(6,4))**

Fungsi dari Matplotlib (plt) untuk membuat wadah gambar (figure) baru dengan ukuran lebar 6 inci dan tinggi 4 inci.

2. **sns.countplot(x='target', data=df)**

Fungsi dari Seaborn (sns) yang digunakan untuk membuat histogram (bar chart) yang menampilkan jumlah (count) observasi untuk setiap nilai unik dalam kolom target. Ini memberikan gambaran visual mengenai distribusi kelas.

3. **plt.title('Distribusi Kelas Target (0 = Tidak Sakit, 1 = Sakit Jantung)')**

Menetapkan judul untuk visualisasi yang dihasilkan, menjelaskan apa arti dari kelas 0 dan 1.

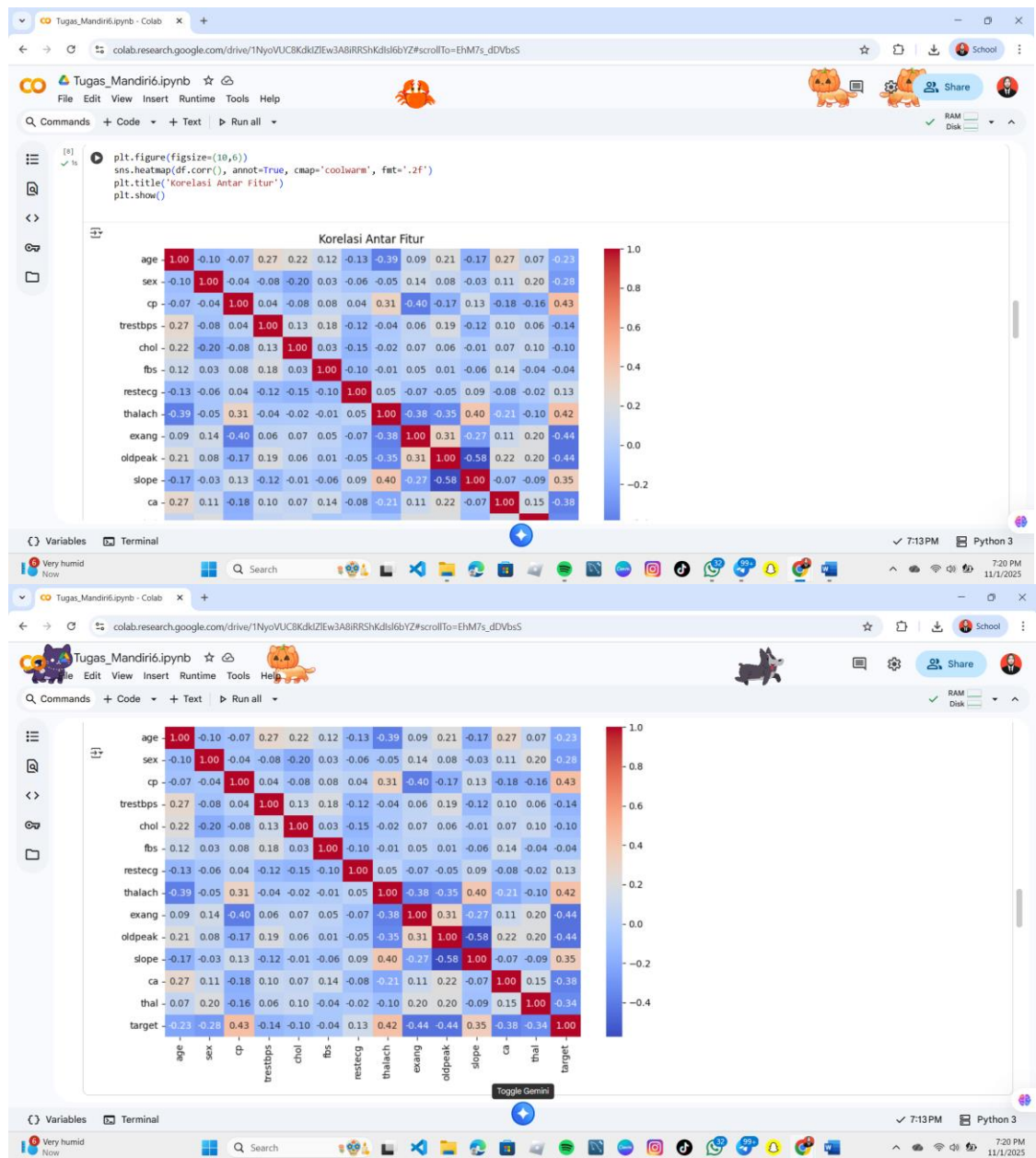
4. **plt.show()**

Menampilkan grafik yang telah dibuat di sel Colab.

Output:

Sebuah Bar Chart berjudul "Distribusi Kelas Target (0 = Tidak Sakit, 1 = Sakit Jantung)".

- Sumbu X (target) Menunjukkan dua kategori: 0 (Tidak Sakit Jantung) dan 1 (Sakit Jantung).
- Sumbu Y (count) Menunjukkan jumlah frekuensi data untuk setiap kategori. Hasil Visualisasi Grafik menunjukkan bahwa jumlah data untuk kelas 0 dan kelas 1 adalah hampir seimbang (kedua bar berada di sekitar angka 500).
- Implikasi (Machine Learning) Karena kedua kelas (Sakit dan Tidak Sakit Jantung) memiliki jumlah data yang seimbang, dataset ini disebut dataset yang seimbang (balanced dataset). Ini adalah kondisi yang baik, karena model klasifikasi (seperti SVM) cenderung tidak akan bias terhadap salah satu kelas.



1. `plt.figure(figsize=(10,6))`

Fungsi dari Matplotlib (`plt`) untuk membuat wadah gambar (`figure`) baru dengan ukuran yang lebih besar (lebar 10 inci dan tinggi 6 inci) agar heatmap terlihat jelas.

2. `df.corr()`

Fungsi Pandas yang menghitung matriks korelasi antara semua pasangan kolom numerik dalam DataFrame `df`. Nilai korelasi berkisar dari -1.0 hingga +1.0.

3. `sns.heatmap(..., annot=True, cmap='coolwarm', fmt='.2f')`

Membuat visualisasi heatmap dari matriks korelasi

4. `plt.title('Korelasi Antar Fitur')`

Menetapkan judul untuk visualisasi.

5. `plt.show()`

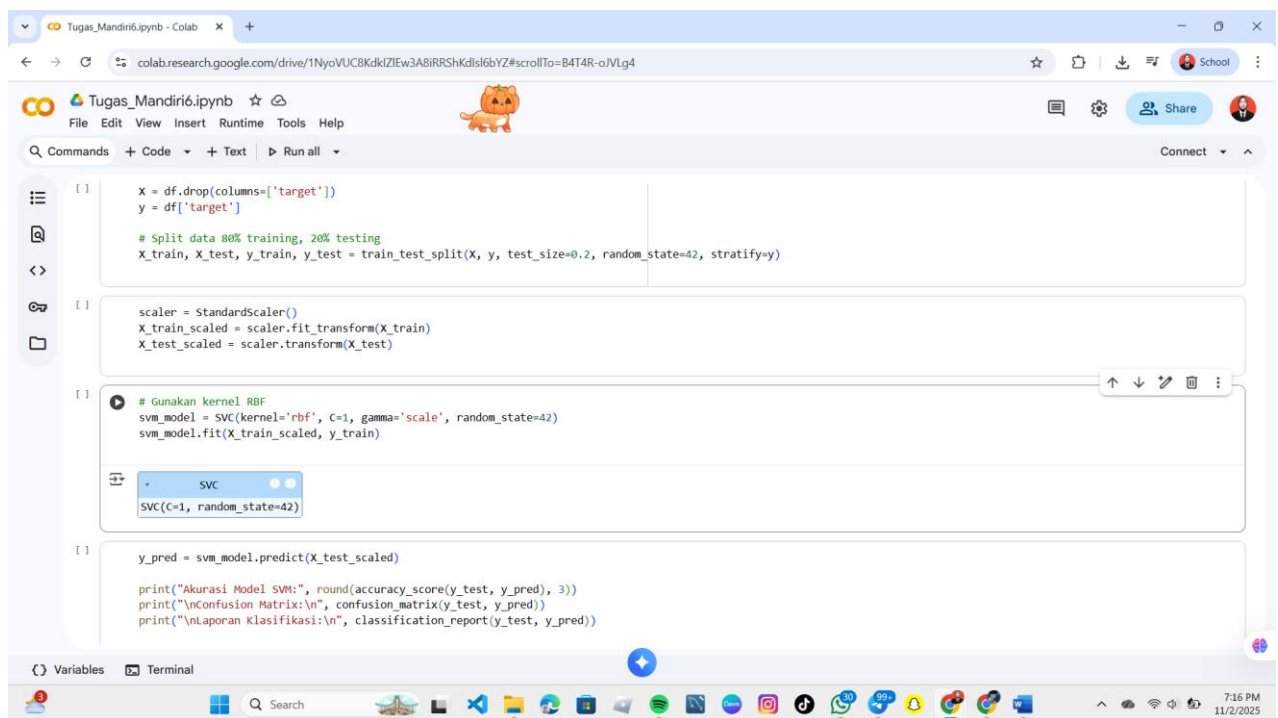
Menampilkan grafik heatmap di sel Colab.

Output:

Matriks Korelasi (Heatmap), memberikan wawasan tentang fitur mana yang paling penting dalam memprediksi penyakit jantung, yang kemudian digunakan oleh model SVM.

Matriks ini menunjukkan koefisien korelasi Pearson antara setiap fitur:

- Warna Merah (Korelasi Positif Kuat): Berarti jika nilai satu fitur meningkat, nilai fitur lainnya cenderung meningkat. (Nilai mendekati +1.0).
- Warna Biru (Korelasi Negatif Kuat): Berarti jika nilai satu fitur meningkat, nilai fitur lainnya cenderung menurun. (Nilai mendekati -1.0).
- Warna Putih/Netral (Korelasi Lemah): Berarti tidak ada hubungan linier yang signifikan. (Nilai mendekati 0.0).



```
[1] x = df.drop(columns=['target'])
    y = df['target']

    # Split data 80% training, 20% testing
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

[2] scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

[3] # Gunakan kernel RBF
    svm_model = SVC(kernel='rbf', C=1, gamma='scale', random_state=42)
    svm_model.fit(X_train_scaled, y_train)

[4] y_pred = svm_model.predict(X_test_scaled)

    print("Akurasi Model SVM:", round(accuracy_score(y_test, y_pred), 3))
    print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("\nLaporan Klasifikasi:\n", classification_report(y_test, y_pred))
```

1. `X = df.drop(columns=['target'])`

Memisahkan Fitur (X): Membuat DataFrame X dengan menghapus kolom target dari data awal (df). X berisi semua variabel independen.

2. `y = df['target']`

Memisahkan Target (y): Membuat Series y yang hanya berisi kolom target (variabel dependen).

3. `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`

Pembagian Data: Membagi data fitur (X) dan target (y) menjadi empat set: Latih (train) dan Uji (test).

- `test_size=0.2`: 20% data digunakan untuk pengujian, 80% untuk pelatihan.

- `random_state=42`: Memastikan hasil pembagian konsisten setiap kali kode dijalankan.

- `stratify=y`: Memastikan proporsi kelas target (y) dalam data latih dan uji tetap sama (penting

karena data seimbang).

4. scaler = StandardScaler()

Inisialisasi Scaler: Membuat objek StandardScaler. StandardScaler mentransformasi data sehingga memiliki rata-rata.

5. X_train_scaled = scaler.fit_transform(X_train)

Scaling Data Latih: Menghitung parameter penskalaan (fit) dari data latih dan menerapkannya (transform).

6. X_test_scaled = scaler.transform(X_test)

Scaling Data Uji: Menerapkan parameter penskalaan yang sama yang dihitung dari data latih ke data uji.

7. svm_model = SVC(kernel='rbf', C=1, gamma='scale', random_state=42)

Inisialisasi Model SVM: Membuat objek model SVC dengan konfigurasi:

- kernel='rbf': Menggunakan Kernel Radial Basis Function, yang efektif untuk klasifikasi data non-linear.

- C=1: Parameter regulasi.

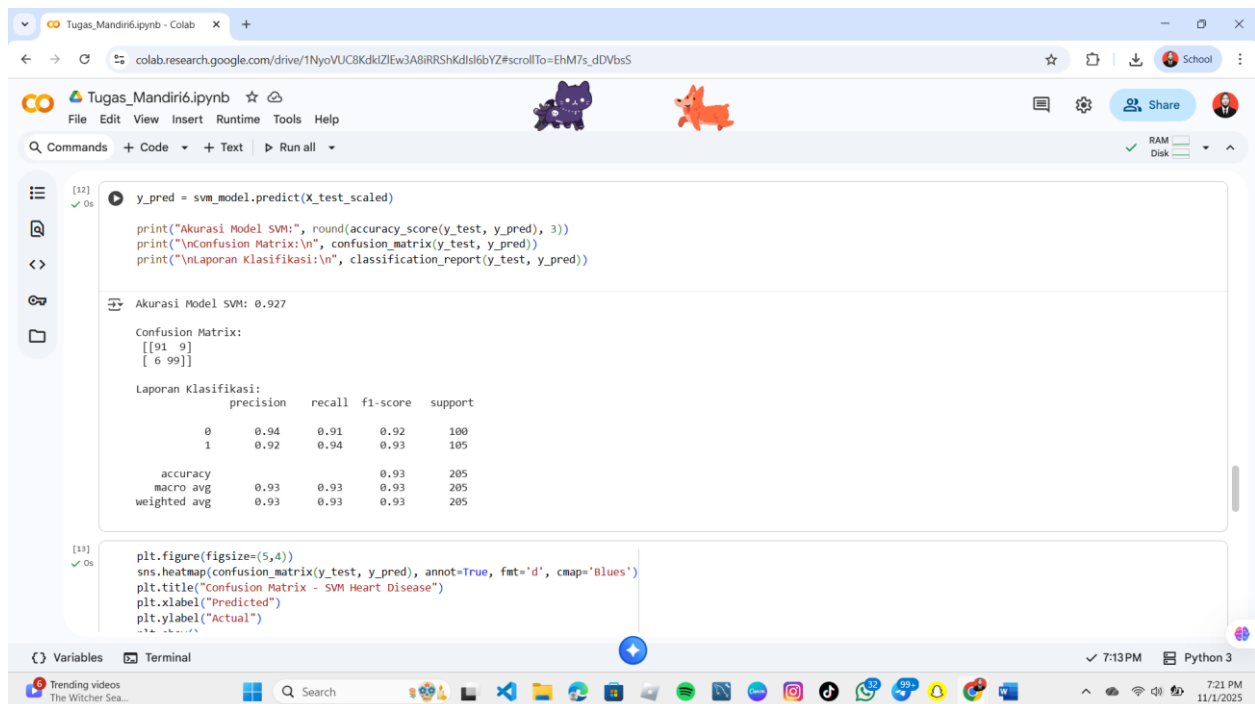
- gamma='scale': Parameter kernel RBF.

8. svm_model.fit(X_train_scaled, y_train)

Pelatihan Model: Melatih model SVM menggunakan fitur latih yang sudah diskalakan (X_train_scaled) dan target latih (y_train).

Output:

SVC(C=1, random_state=42) (Output ringkasan objek model).



1. `y_pred = svm_model.predict(X_test_scaled)`

Prediksi: Menggunakan model SVM yang sudah dilatih (`svm_model`) untuk memprediksi label kelas pada data uji yang sudah diskalakan (`X_test_scaled`). Hasil prediksi disimpan di variabel `y_pred`.

2. `print("Akurasi Model SVM:", round(accuracy_score(y_test, y_pred), 3))`

Akurasi: Menghitung tingkat akurasi model. Akurasi adalah proporsi total prediksi yang benar terhadap total data uji. Hasilnya dibulatkan hingga 3 angka desimal.

3. `print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))`

Confusion Matrix: Menghitung dan menampilkan Confusion Matrix. Matriks ini adalah tabel ringkasan kinerja algoritma klasifikasi.

4. `print("\nLaporan Klasifikasi:\n", classification_report(y_test, y_pred))`

Classification Report: Menghitung dan menampilkan laporan rinci yang berisi metrik Precision, Recall, dan F1-Score untuk setiap kelas (0 dan 1).

Output:

- Akurasi Model SVM: 0.927

Model SVM ini memiliki tingkat akurasi sebesar 92.7%. Ini berarti model memprediksi status penyakit jantung dengan benar untuk 92.7% dari total pasien dalam data uji. Nilai ini menunjukkan kinerja model yang sangat kuat.

- Confusion Matrix: [[91 9] [6 99]]

Matriks ini merangkum hasil prediksi pada 205 sampel data uji:

Actual \ Predicted

0 (Tidak Sakit)

1 (Sakit Jantung)

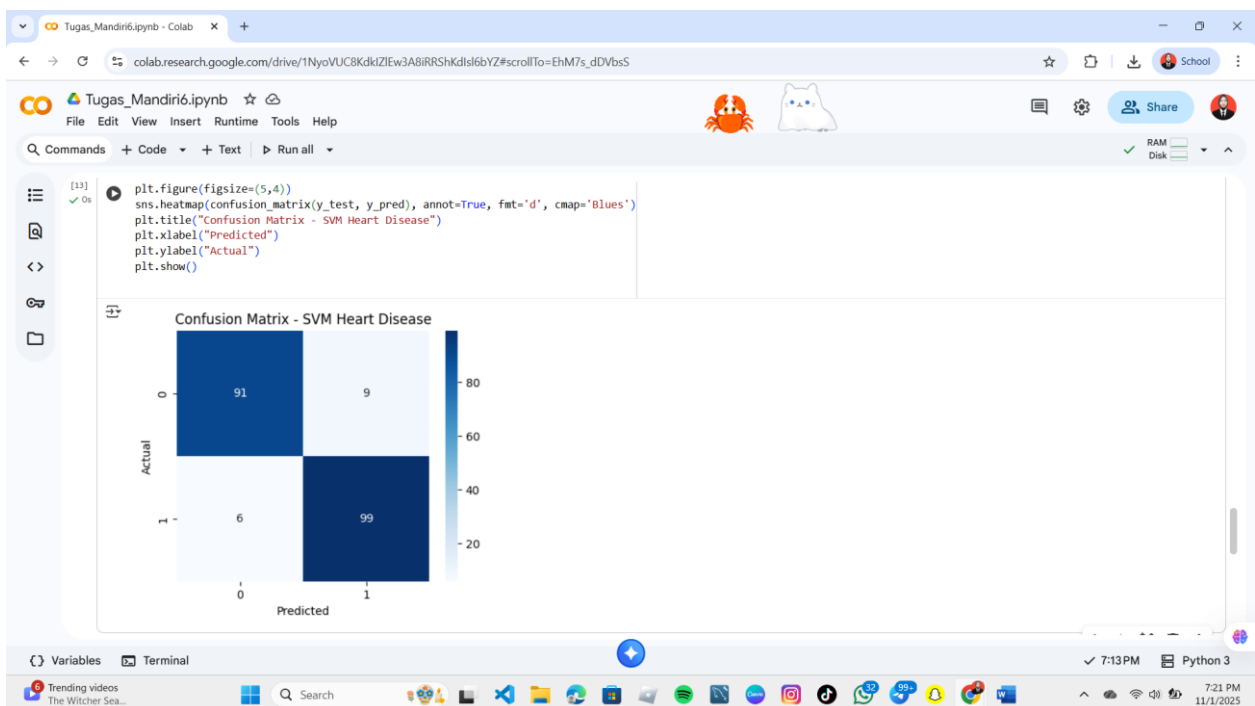
0 (Tidak Sakit): 91 (True Negative) dan 6 (False Negative)

1 (Sakit Jantung): 9 (False Positive) dan 99 (True Positive)

- True Positive (TP = 99): Model dengan benar memprediksi 99 pasien Sakit Jantung.
- True Negative (TN = 91): Model dengan benar memprediksi 91 pasien Tidak Sakit Jantung.
- False Negative (FN = 6): Model gagal mendeteksi 6 pasien yang sebenarnya Sakit Jantung (kesalahan Tipe II, paling berisiko dalam diagnosis medis).
- False Positive (FP = 9): Model secara salah memprediksi 9 pasien yang sebenarnya Sehat menjadi Sakit Jantung (kesalahan Tipe I).

Laporan Klasifikasi:

- Precision (Presisi): Seberapa akurat model ketika memprediksi kelas tersebut. (Contoh: Ketika model memprediksi kelas 0, ia benar 94% dari waktu).
- Recall (Sensitivitas): Seberapa baik model menemukan semua kasus positif yang relevan. (Contoh: Model menemukan 94% dari total kasus Sakit Jantung yang ada).
- F1-Score: Rata-rata harmonik dari Precision dan Recall.



1. plt.figure(figsize=(5,4))

Fungsi Matplotlib untuk membuat wadah gambar dengan ukuran yang ditentukan.

2. confusion_matrix(y_test, y_pred)

Fungsi dari sklearn.metrics yang menghitung matriks perbandingan antara label target yang sebenarnya (y_{test}) dan label yang diprediksi oleh model (y_{pred}).

3. sns.heatmap(..., annot=True, fmt='d', cmap='Blues')

Fungsi Seaborn untuk membuat visualisasi heatmap dari Confusion Matrix.

4. `plt.title(...)`, `plt.xlabel(...)`, `plt.ylabel(...)`

Menetapkan judul grafik serta label untuk sumbu X (Predicted) dan sumbu Y (Actual).

5. `plt.show()`

Menampilkan grafik heatmap yang sudah selesai.

Output:

sebuah Heatmap yang secara visual merangkum kinerja model SVM dalam memprediksi status penyakit jantung.

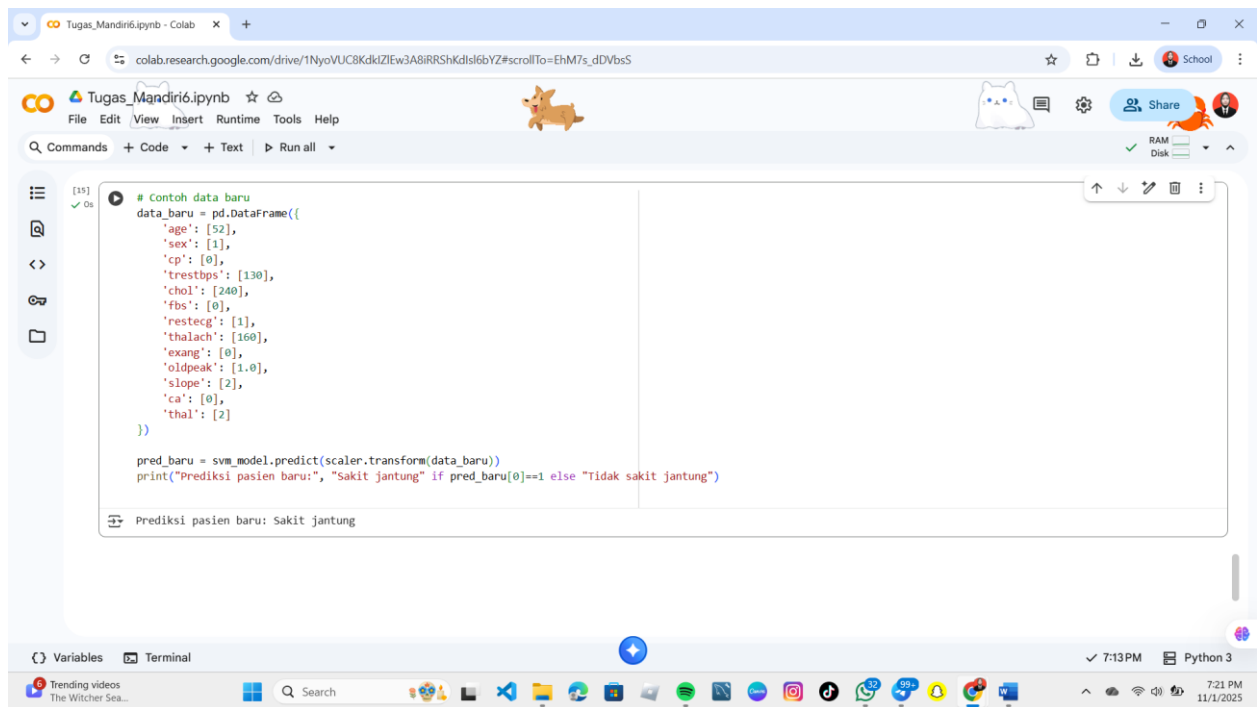
Interpretasi Hasil

1. Diagonal Utama (Prediksi Benar): Angka 91 (kiri atas) dan 99 (kanan bawah) adalah prediksi yang benar. Ini menunjukkan model benar dalam 190 kasus (91 + 99).

1. Diagonal Sekunder (Prediksi Salah):

- 9 (False Positive / Kesalahan Tipe I): Model salah memprediksi 9 pasien sehat sebagai pasien sakit.
- 6 (False Negative / Kesalahan Tipe II): Model gagal mendeteksi 6 pasien sakit (memprediksi mereka sehat). Dalam konteks medis, ini adalah jenis kesalahan yang paling kritis.

Kesimpulan: Visualisasi ini dengan jelas menunjukkan bahwa model SVM memiliki kinerja yang kuat dengan jumlah kesalahan (15 kasus) yang minim.



```
[15]: # Contoh data baru
data_baru = pd.DataFrame({
    'age': [52],
    'sex': [1],
    'cp': [0],
    'trestbps': [130],
    'chol': [240],
    'fbs': [0],
    'restecg': [1],
    'thalach': [160],
    'exang': [0],
    'oldpeak': [1.0],
    'slope': [2],
    'ca': [0],
    'thal': [2]
})

pred_baru = svm_model.predict(scaler.transform(data_baru))
print("Prediksi pasien baru:", "Sakit jantung" if pred_baru[0]==1 else "Tidak sakit jantung")

Prediksi pasien baru: Sakit jantung
```

1. `data_baru=pd.DataFrame({...})`

Pembuatan Data Sampel Baru: Membuat satu baris data (sebagai Pandas DataFrame) yang mewakili fitur-fitur klinis satu pasien baru. Setiap key dalam dictionary (misalnya age, sex) adalah nama fitur yang sama dengan data latih.

2. `scaler.transform(data_baru)`

Penskalaan Data Baru (Wajib): Menerapkan fungsi transform dari objek scaler (StandardScaler yang dilatih pada data latih) ke data pasien baru. Ini sangat penting karena model SVM dilatih menggunakan nilai yang sudah diskalakan. Data baru harus memiliki skala yang sama agar prediksi akurat.

3. `pred_baru = svm_model.predict(...)`

Prediksi: Menggunakan model SVM yang sudah dilatih (svm_model) untuk memprediksi status penyakit jantung pasien baru berdasarkan fitur yang sudah diskalakan. Hasil prediksi (0 atau 1) disimpan di variabel pred_baru.

4. `print("Prediksi pasien baru: {}".format(...))`

Interpretasi dan Output: Menampilkan hasil prediksi dalam kalimat yang mudah dipahami:

- Jika pred_baru bernilai 1, outputnya adalah "Sakit Jantung".
- Jika pred_baru bernilai 0, outputnya adalah "Tidak Sakit Jantung".

Output:

- Input Pasien Baru: memasukkan fitur pasien dengan beberapa karakteristik, termasuk usia 52 tahun, detak jantung maksimum (thalach) 160, dan tidak ada angina akibat olahraga (exang=0).
- Prediksi Model: Berdasarkan kombinasi fitur yang dimasukkan dan pola yang dipelajari selama pelatihan, model Support Vector Machine (SVM) memprediksi bahwa pasien ini cenderung Sakit Jantung (kategori 1).

Kesimpulan hasil implementasi algoritma: Berdasarkan implementasi yang telah dilakukan, model Support Vector Machine (SVM) dengan Kernel RBF berhasil dilatih untuk klasifikasi status penyakit jantung, menunjukkan kinerja yang sangat kuat dan seimbang pada data pengujian. Model mencapai Akurasi 92.7%, yang didukung oleh Confusion Matrix yang mencatat hanya 15 kesalahan klasifikasi dari 205 sampel uji (6 False Negatives, yang kritis dalam diagnosis). Selain itu, Recall dan Precision yang sama-sama tinggi (di atas 0.92) untuk kedua kelas mengonfirmasi bahwa model tidak bias dan sangat efektif dalam memprediksi pasien yang sakit maupun yang sehat.

LINK DATASET KAGGLE: <https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>

LINK GITHUB UPLOAD TUGAS : <https://github.com/Yurida26/Machine-Learning/tree/2e12931d22ee394ed2e32b7a74000f3ef089a25d/Praktikum6>

